EVA

Local Voice and Avatar Project Documentation

Version 1.3 (documentation)

Daniel Ocks

29 October 2026

Contents

1 Cheat sheet **2**

2 Knowledge gained **4**

3 Open issues **8**

4 Others: project and maintenance notes **10**

A Package versions and model settings **11**

B Sources and template **12**

Code examples

1 Check the PC and install the backend 2
2 Run the setup check 2
3 Current pinned Python packages 11

1 Cheat sheet

Page 1 of 2: set up EVA. EVA is a local character you can talk to in Unity. Choose English or German before speaking. Installation needs Internet; the downloaded speech and answer models run on your PC.

What you need

Software	Windows x64, Python **3.11 x64**, Unity **6000.3.12f1**, Unity Hub, Git with Git LFS, and Ollama.
Graphics card	An NVIDIA card with CUDA and FP16 support and a suitable driver. This version cannot run on the CPU alone.
Previous PCs	Desktop: RTX 5070 Ti, 16 GB graphics memory, Ryzen 7 9700X, 32 GB RAM. Laptop: RTX 5070 Laptop GPU with about 8 GB graphics memory.
Suggested PC	Estimate: a recent 6-core CPU, 16 GB RAM (32 GB recommended), an NVIDIA RTX GPU with 8 GB graphics memory (12–16 GB recommended), and an SSD.
Space and limits	Start with at least 12 GB free for backend setup. Unity, assets and the import cache need extra space. These are estimates, not measured minimum requirements. Run the project checks on the actual PC.

1. Copy the project and install the software

Keep `Assets`, `Packages`, `ProjectSettings`, `Backend` and the project instructions. Create a new `Backend/.venv` and let Unity create a new `Library`. Keep existing models and voices. A copied Python environment may cause *No Python at* / exit code 103.

Open PowerShell in **Backend**:

```
Set-ExecutionPolicy -Scope Process Bypass
.\hardware_report.ps1
# Read hardware-report.json. Check the GPU first.
.\setup_backend.ps1 -RecreateEnvironment -InstallOllama
```

Listing 1: Check the PC and install the backend

The NVIDIA setup creates a Python environment, installs the pinned packages and checks or downloads Whisper Small, Llama 3.2 3B and all four Piper voices. It supplies CUDA runtime libraries; a full CUDA developer toolkit is normally unnecessary. Appendix A of the full report lists the package versions.

2. Check the backend

```
.\.venv\Scripts\python.exe verify_backend.py
```

Listing 2: Run the setup check

The last line must say:

```
[EVA Check] SUCCESS: The local GPU backend is ready.
```

For missing Microsoft runtime DLLs, install the Visual C++ x64 Redistributable. See the ONNX Runtime requirements.

3. Open Unity

Use Unity **6000.3.12f1** (recommended). Newer versions may work but are untested. Open `Assets/_Project/Scenes/Main.unity`, wait for import and fix compiler errors before Play. This is also the configured build scene. The launcher starts or reuses Ollama and Python. Normal installations need no manual path overrides or extra PowerShell server windows.

Cheat sheet — page 2 of 2: choose, talk and test

Choose your EVA after loading

Press Play and wait for loading. Input stays blocked until the backend sends its readiness message. Then use the selection menu:

1. **01 CHARACTER:** choose one of six designs. Only one EVA is visible.
2. **02 BEHAVIOUR:** choose Neutral, Friendly or Expressive body animations.
3. **03 LANGUAGE:** choose English or Deutsch. This fixes recognition, answer text and speech to that language. There is no automatic language switching.
4. **04 VOICE:** click a name to select it, independently of the design. The small play icon previews that voice without selecting it; click again to stop.

Select **Start conversation**. Confirmed choices are saved locally, with a separate voice choice per language. A preview stops before conversation starts.

Voice	English	German
Female	Kristin, 22,050 Hz	Ramona, 16,000 Hz
Male	Mike, 22,050 Hz	Thorsten-high, 22,050 Hz

Hz is the audio sample rate: samples of the sound wave per second. For example, 16,000 Hz means 16,000 samples per second. Unity uses the supplied rate so speech plays at the correct speed and pitch.

Desktop controls

Before Play, choose EVA > Controls > Desktop (WASD + Mouse). After starting conversation, click empty Game-view space to capture the mouse. **WASD**: move; **mouse**: look; **Shift**: move faster; **Esc**: release the mouse. Hold **B** to speak, then release to send. A new recording interrupts the old answer. Press **M** to reopen settings when EVA is idle. Change language there before speaking another language.

Text boxes start hidden. Press **T** to show/hide the recognized question and EVA's answer. Both reveal words as you speak or EVA generates a reply. With the mouse released, use their move and resize handles.

Meta Quest 3 controls

Start Quest Link or Air Link first; choose EVA > Controls > VR (Meta Quest). In the menu, use the **right stick** and **A** to navigate/confirm. Right **B** opens settings while idle or closes the menu with Start/Continue. During conversation, left stick moves; right stick snap-turns by 30 degrees. Hold right **A** to record and release to send; release the menu confirmation press first. Left **X** shows/hides text; left **Y** repositions the surfaces. Menu and text sit near EVA, about 2–3 m away, and stay in the scene when you turn your head. In VR, text starts visible. Adjust size/distance in `Assets/_Project/Resources/EVAControlSettings.asset`, under VR panels (metres). Unity and Python run on the PC, not the headset.

Quick checks and fixes

Try both menu languages and both voices in each. Listen to previews and complete answers; check lip-sync, styles, gaze, text boxes and interrupting speech. Repeat with the real microphone and headset before a demo.

CUDA/DLL error: rerun setup with the project Python; do not mix CPU and GPU ONNX Runtime packages.

No connection: read `Backend/backend-runtime.log` and the Console. Stop Play, fix the cause and restart. For exit 103, recreate the environment.

VR closes: start Quest Link first and fully restart Unity.

Missing speech: long answers can exceed the scene's 60-second audio queue.

2 Knowledge gained

2.1 How EVA works now

EVA is a local voice application with one visible character in Unity. After loading, the user chooses a design, body-animation style, language and voice. The six designs share the same conversation system. Extra scene hosts, if present, are disabled during Play; three visible agents are not required.

The **backend** is the Python program behind Unity. A **WebSocket** is the local connection carrying recordings, text, audio and mouth timings. The **GPU** is the graphics card; it runs the models and draws the scene.

1	**Unity records speech** while B or the VR recording button is held.
2	**Python receives the recording** with the selected language and voice profile.
3	**Faster-Whisper creates text** using the selected language explicitly.
4	**Ollama runs Llama 3.2** with a language instruction for this answer.
5	**Piper creates speech and timings** with the selected matching voice.
6	**Unity plays audio and animates EVA**, including mouth shapes and body states.

Figure 1: The current path from a question to a spoken answer.

Unity sends small audio pieces while the recording button is held. Whisper returns provisional text, which may correct recent words. Releasing the button finishes recognition and starts the answer. Answer text and audio arrive in small pieces, so playback can begin before the whole answer is ready. Both text boxes reveal complete words. Desktop text starts hidden and can be shown with T; VR text starts visible and can be shown or hidden with left X.

2.2 Tools and reasons for using them

Tool	Job and reason
Unity and C#	Character display, microphone capture, menu, desktop/VR controls, animation, audio and text boxes in one application.
Python	Connects the speech tools, manages conversation steps and sends results to Unity.
Faster-Whisper / CTranslate2	Local speech recognition. Whisper Small runs on CUDA with FP16 to use the NVIDIA card.
Ollama / Llama 3.2 3B	Ollama loads the local language model; Llama writes answers. The 3B model limits memory use compared with larger models.
Piper / ONNX Runtime	Local English/German speech. ONNX Runtime runs the voice models on CUDA; Piper's phoneme timings support lip-sync.
Microsoft Rocketbox	Existing rigged avatars, body animations and facial shapes, including the 15 visemes used by EVA.
Input System / NativeWebSocket	Input handling and the persistent Unity–Python connection.
OpenXR / Quest Link	PC VR with Meta Quest 3 while Unity and models run on Windows.

These choices support the current pipeline. Official tool links are in Appendix B.

2.3 Menu, language and voice

The menu provides six character designs and three body styles: Neutral, Friendly and Expressive. The styles change body animation, not the language model's personality. The states remain Idle, Listening, Thinking and Speaking.

03 LANGUAGE fixes the whole turn. English/Deutsch is sent to Whisper, the answer instruction and the voice selector. There is no automatic language detection or switching. Speak in the selected language and reopen settings with M, or right B in VR, to change it while idle.

04 VOICE is independent of the design. English offers Kristin/female and Mike/male; German offers Ramona/female and Thorsten/male. Clicking a name selects it. Its small play icon only previews that voice. Loading and stop symbols replace the play icon while needed. Preview audio uses a separate AudioSource, does not alter the conversation and does not animate the mouth. It stops on language/voice changes, conversation start or menu closure.

2.4 VR interface and controls

In VR, screen panels close to the eyes were hard to read at the edges of the view. The interface now uses **World Space**: the menu sits beside EVA, conversation text sits below the face, and loading appears in front of the user. The surfaces are usually about 2–3 m away. They keep their position in the scene when the user turns their head, so they can be viewed by looking towards EVA.

Use the right stick and A to select menu controls. Right B opens settings while idle or closes the menu with Start/Continue. During conversation, hold right A to record and release it to

send. Release A after confirming a menu choice first. The left stick moves and the right stick snap-turns by 30 degrees. Left X toggles conversation text; left Y repositions the surfaces from the current position. Opening the menu also repositions it. Head tracking remains active during menus, while recording and walking stay blocked until loading and selection finish.

Distance, width and offsets can be changed in `Assets/_Project/Resources/EVAControlSettings.asset`, under VR panels (metres). Press Y after changing values during Play. Save the asset outside Play to keep changes. Desktop layout and VR text visibility are stored separately. Check readability in both eyes and while standing or seated.

2.5 Problems encountered and lessons learned

Area	What went wrong	Current solution or lesson
GPU dependencies	A CUDA DLL was missing; a CPU-only ONNX package also replaced the GPU runtime.	Pin the NVIDIA packages, protect the GPU runtime in setup and use the shared DLL configuration in `cuda_runtime.py`. Run preflight.
PC transfer	A copied environment pointed to the previous PC's Python and exited with code 103.	Install Python 3.11 x64 locally and recreate only `.venv`. Existing models and voices can stay.
Language choice	Automatic recognition, answer language and voice could disagree.	Use one explicit menu language for all three stages. Piper itself does not detect a text's language.
Complete playback	Speech ended early or filled the queue.	Wait for audio playback to finish. The remaining 60-second queue limit is documented separately.
Lip-sync	The wrong face mesh had no visemes; mouth motion was too strong.	Update face references when changing design and keep the scene's maximum viseme weight at 50%.
Voice vocabulary	Ramona's older model lacks some symbols produced by newer eSpeak.	`piper_compat.py` filters unsupported phonemes before model input and alignment. A regression test checks supported input is preserved.
Rig and turning	Bone axes differ; full-body turns could move EVA away.	Keep import-time bone fixes, calibrated head axes and leg-only turn animation.
Server lifecycle	An old backend could remain after a script reload and fail new connections.	Clean up owned process trees on disable/reload; wait for `server.ready` and retain server tracebacks.
Preview UI	A new play icon lacked a CanvasRenderer and failed during raycasts.	Require the component and add it explicitly before creating the icon. Both creation paths and all icon states passed an isolated Unity test.

2.6 Lip-sync and warm-up

A **phoneme** is a speech sound; a **viseme** is a visible mouth shape. Rocketbox provides 14 speech groups plus silence. Piper supplies phoneme timings, Python maps them to these shapes, and Unity blends them against the audio clock. No new lip-sync model was trained.

Warm-up prepares the first model calls: Whisper processes silence, Ollama writes a short reply and the four Piper voices synthesize sample text. This loads model resources and checks GPU execution; it does not train or fine-tune the models. Input stays blocked until startup and the selection step are complete.

3 Open issues

This section lists current limits and checks still needed. The copied-environment failure, fixed-language selection, stale-process cleanup and missing preview renderer have implemented fixes; they are not unfinished features.

3.1 Current limits

Long answers can exceed the audio queue

The saved scene buffers at most 60 seconds of queued audio. An incoming piece that would exceed the limit is skipped with an error. Python does not yet slow down to match playback. A long answer can therefore lose audio even though normal short answers play fully.

Connection loss after startup needs a restart

The launcher retries during startup and waits for `server.ready`. After startup there is no continuous reconnect process or full health check of every model service. If the connection is lost, fix the cause, stop Play and start it again. Check the Console and `Backend/backend-runtime.log`.

Recognition and answer quality need real-user tests

Language is fixed by the menu, so the user must speak that language. This removes automatic switching but does not prevent misheard words. The synthetic German test produced transcription errors. Real microphone recordings, accents, noise, silence and short questions still need systematic checks.

The 3B language model can misunderstand or invent facts. It has not been fine-tuned for EVA. There is no hard output-token limit or reliable short-answer target. A language instruction is also not a separate check of the generated text.

VR and shared graphics memory

Unity rendering, recognition, four speech models and the answer model share one GPU. A laptop with less graphics memory may have less possibilities. Test actual memory use, first-audio delay and frame rate. Desktop movement keeps a fixed camera height and has no collision-based walking.

VR uses PC OpenXR through Quest Link/Air Link. It needs the correct runtime, headset microphone and sound output. It is not a standalone Android/Quest app. The menu supports controller navigation, but the complete current flow still needs a real-headset acceptance run.

3.2 Manual acceptance checks still needed

1. **Start/stop and reload:** repeat at least five times, including while loading or speaking. Check owned-process cleanup and readiness/input gating.
2. **Menu and previews:** try designs, styles, both languages and voices. Preview the unselected voice without changing the choice; cancel while loading, switch voices and start a conversation. No old preview should start later.
3. **Real conversation:** speak in each selected language with a real microphone. Check transcript accuracy, answer quality, voice, complete playback, lip-sync, interruption and text-box controls.

The automated checks described in Section 2 support the backend and source changes. They do not replace these UI, microphone and headset checks.

4 Others: project and maintenance notes

4.1 Important files

Paths start at the project root. Keep the complete prepared backend folder when copying or packaging the application.

File or folder	Purpose
`README.md`, `Backend/BACKEND_SETUP.md`	Current usage, handover instructions and backend setup.
`Backend/server.py`	The single speech/conversation server entry point.
`Backend/cuda_runtime.py`	Shared NVIDIA DLL setup for server and preflight.
`Backend/voice_config.py`, `Backend/piper_compat.py`	Four voice models and the phoneme-vocabulary compatibility fix.
`Backend/setup_backend.ps1`, `Backend/verify_backend.py`	Environment, package/model installation and GPU checks.
`Assets/_Project/Scripts/`	Runtime code for menu, preview icons, loading, input, audio, text, character animation and VR.
`Assets/Editor/`, `Assets/_Project/Editor/`	Avatar import, rig/VR setup and validation. Keep these for correct asset reimport on another PC.
`Assets/_Project/Scenes/Main.unity`	Saved application scene and build target.
`Assets/_Project/Resources/EVAControlSettings.asset`	Desktop/VR mode and VR panel distance, sizes, offsets and default text visibility.
`Packages/manifest.json`, `Packages/packages-lock.json`	Unity package requirements and resolved versions.

4.2 Testing and troubleshooting

`verify_backend.py` checks packages, CUDA and model files. The 14 checks in `test_response_language.py`, `test_voice_preview.py` and `test_streaming_transcription.py` use simulated models and do not record audio. Run `test_voice_profiles.py` outside Play for real voice synthesis and reusable test WAVs. It also exercises Ramona's phoneme compatibility fix.

With the backend already ready, `test_live_voice_preview.py` checks all four short previews, cached results, audio formats and signal. Then `test_live_voice_profiles.py` uses the generated WAVs to check the complete Whisper/Ollama/Piper path. These clients use their own connections and do not play test audio through Unity.

Unity's EVA > Interface > Validate Selection Menu checks references in Edit mode. In Play, with the menu open, it checks all 72 design/style/language/voice combinations, recording gates and both preview renderers. Stop a running preview before this check. Other retained appearance/gaze tools remain under EVA > Appearance, EVA > Gaze and EVA > Diagnostics.

Errors: `Backend/backend-runtime.log` (1 MB plus one backup). Stop Play before edits. Unity closes only the process trees it started.

A Package versions and model settings

Install with the setup script. It reads `Backend/requirements.txt` and protects the GPU ONNX Runtime from being replaced by the CPU package.

```
annotated-types==0.8.0
anyio==4.15.1
certifi==2026.7.22
click==8.5.0
colorama==0.4.6
filelock==3.32.5
flatbuffers==25.12.19
fsspec==2026.7.0
h11==0.16.0
hf-xet==1.6.0
httpcore==1.0.9
httpx==0.28.1
huggingface-hub==1.30.0
idna==3.19
ml-dtypes==0.6.0
numpy==2.4.6
nvidia-cublas-cu12==12.9.2.10
nvidia-cuda-nvrtc-cu12==12.9.86
nvidia-cuda-runtime-cu12==12.9.79
nvidia-cudnn-cu12==9.25.1.1
nvidia-cufft-cu12==11.4.1.4
nvidia-nvjitlink-cu12==12.9.86
av==18.1.0
ctranslate2==4.8.2
faster-whisper==1.2.1
ollama==0.6.2
onnx==1.22.0
onnxruntime-gpu==1.26.0
packaging==26.3
pathvalidate==3.3.1
piper-tts==1.8.0
protobuf==7.36.1
pydantic==2.13.5
pydantic-core==2.46.5
PyYAML==6.0.3
tokenizers==0.23.2
tqdm==4.70.0
typing-extensions==4.16.0
typing-inspection==0.4.4
websockets==17.0.1
```

Listing 3: Current pinned Python packages

Unity packages used by EVA

Package	Version
Unity editor	6000.3.12f1
Universal Render Pipeline	17.3.0
Input System / uGUI	1.19.0 / 2.0.0
XR Management / OpenXR	4.5.3 / 1.16.1
NativeWebSocket	Git package, branch `upm-2`
Saved Git revision	`c612a4fef60f2ae57614b73202d2d261ba56aa3e`

Keep `Packages/manifest.json` and `Packages/packages-lock.json` together.

Model and voice settings

- **Recognition:** Whisper `small` from `Systran/faster-whisper-small`; CTranslate2, CUDA device 0, FP16. The menu's `en` or `de` is passed explicitly. Automatic language detection is disabled.
- **Answers:** local Ollama model `llama3.2:3b`, 4,096-token context. The server requires at least 99% reported GPU allocation. `ollama==0.6.2` is the Python client version, not the desktop application version.
- **Female voices:** `en_US-kristin-medium` and `de_DE-ramona-low`.
  Male voices: `en_US-mike-medium` and `de_DE-thorsten-high`.
  Each needs its `.onnx` model and `.onnx.json` configuration. Downloads come from `rhasspy/piper-voices`; menu choices select the profile.

ONNX is a model file format; ONNX Runtime is the software that runs it. CUDA/cuDNN libraries are installed as Python packages. A suitable NVIDIA driver is still required. The shared `cuda_runtime.py` prepares DLL paths for both server and preflight. No extra Python dependency was added by this cleanup.

B Sources and template

B.1 Current project evidence

The source review of **28 October 2026** covered the selection menu, fixed language, independent voices, inline previews and code cleanup. This revision also includes live text, VR controls and panel placement from **29 October 2026**. It uses the actual scripts and saved Main scene. Scene settings can differ from script defaults.

`reference/source-inventory.json` records source paths and SHA-256 checksums from the 28 October baseline; it is not a checksum list of later edits. A checksum identifies the reviewed file contents. The Python package snapshot comes from `Backend/requirements.txt`; Unity versions come from the package files and `ProjectVersion.txt`.

`reference/pc-backend-validation-2026-10-28.md` records the current desktop, resolved startup/renderer problems, compilation, automated checks and live voice/preview tests. Detailed local logs are excluded from Git. The tests used generated speech, not the Windows microphone or a headset. They do not establish exact transcription accuracy, answer quality, frame rate or a complete Player build.

Development problems in Section 2 are lessons from the code, tests and project history. Resolved problems are labelled as such; remaining limits are in Section 3.

B.2 Official tool documentation

These upstream pages explain the components. Their current versions do not replace EVA's pinned package list or prove its performance.

U1 Faster-Whisper: speech recognition and GPU dependencies.

U2 Piper: phoneme timing for alignment and lip-sync.

U3 ONNX Runtime: CUDA execution and native libraries.

U4 Ollama: local chat API.

U5 Llama 3.2 3B: model information and license links.

U6 Microsoft Rocketbox: models, animation and license.

U7 Piper engine and license; voice cards: Kristin, Mike, Ramona, Thorsten-high.

B.3 Template and editing

The document retains Overleaf's Technical Document Template and its unchanged `ol-softwaremanual.cls`. It supplies Roboto fonts and the title page. Template credits are in `TEMPLATE_NOTES.md`.

Open `Documentation_EN` in Visual Studio Code. Edit `main.tex`, `setup_guide.tex` or the relevant file in `sections`. Save, then press Ctrl+Shift+B to rebuild all three PDFs. The cheat sheet shares its source with Section 1 of this report. Code examples use `listings`; shell escape, Python/Pygments and Perl are not required by the LaTeX build.